Tutorial: Introduction to JShell

Overview

JShell is an interactive tool introduced in JDK 9 that allows developers to run Java code snippets and see immediate results. It's a powerful tool for learning, exploring, and prototyping Java code without the need to create a complete Java application.

Starting JShell

To start JShell, you need to have JDK 9 or later installed. You can download the latest JDK from the Oracle website.

Starting JShell

1. **Open Terminal (Mac/Linux) or Command Prompt (Windows)**
2. **Type `jshell` and press Enter**

```
$ jshell
```

You will see the JShell prompt:

```
|  Welcome to JShell -- Version 11.0.9
|  For an introduction type: /help intro

jshell>
```

Basic Commands

JShell provides several commands to help you work with the tool efficiently. Commands in JShell start with a forward slash (`/`).

Useful Commands

- **/help**: Displays help information.
- **/vars**: Lists all variables.
- **/methods**: Lists all methods.
- **/list**: Lists all code snippets entered.
- **/edit**: Opens a code snippet in an editor for modification.
- **/reset**: Resets the JShell state.
- **/exit**: Exits JShell.

Example: Basic Commands

```
jshell> /help
|  Type a Java language expression, statement, or declaration.
|  Or type one of the following commands:
|  /list [<name>|<id>|-all|-start]
|       list the source you have typed
|  /edit <name>|<id>
|       edit a source entry referenced by name or id
|  /drop <name>|<id>
|       delete a source entry referenced by name or id
|  /save [-all|-history|-start] <file>
|       save snippet source to a file
|  /open <file>
|       open a file as source input
|  /vars [<name>|<id>|-all|-start]
|       list the declared variables and their values
|  /methods [<name>|<id>|-all|-start]
|       list the declared methods and their signatures
|  /types [<name>|<id>|-all|-start]
|       list the declared types
|  /imports
|       list the imported items
|  /exit
|       exit jshell
|  /env [-class-path <path>] [-module-path <path>] ...
|       view or change the evaluation context
|  /reset [-class-path <path>] [-module-path <path>] ...
|       reset jshell
|  /reload [-restore] [-quiet] ...
|       reset and replay relevant history -- current or previous (-
restore)
|  /history [-all]
|       history of what you have typed
|  /help [<command>]
|       get information about a jshell command
|  /? [<command>]
|       get information about a jshell command
|  /!
|       rerun last snippet
|  /<id>
|       rerun snippet by id
```

Basic Operations in JShell

Declaring Variables

You can declare variables just like you would in a Java program.

```
jshell> int number = 10
number ==> 10
```

```
jshell> String message = "Hello, JShell!"
message ==> "Hello, JShell!"

jshell> boolean flag = true
flag ==> true
```

Performing Calculations

You can perform calculations and see the results immediately.

```
jshell> int a = 5
a ==> 5

jshell> int b = 10
b ==> 10

jshell> int sum = a + b
sum ==> 15

jshell> double pi = 3.14
pi ==> 3.14

jshell> double area = pi * 5 * 5
area ==> 78.5
```

Methods and Functions

You can define and invoke methods directly in JShell.

Example: Defining a Method

```
jshell> int add(int x, int y) {
   ...>     return x + y;
   ...> }
|  created method add(int,int)

jshell> int result = add(3, 4)
result ==> 7
```

Loops and Conditionals

You can use loops and conditional statements just like in a regular Java program.

Example: Using a Loop

```
jshell> for (int i = 1; i <= 5; i++) {
   ...>     System.out.println("Count: " + i);
   ...> }
Count: 1
Count: 2
Count: 3
Count: 4
Count: 5
```

Example: Using an If-Else Statement

```
jshell> int number = 10
number ==> 10

jshell> if (number > 5) {
   ...>     System.out.println("Number is greater than 5");
   ...> } else {
   ...>     System.out.println("Number is 5 or less");
   ...> }
Number is greater than 5
```

Working with Classes

You can define and instantiate classes in JShell.

Example: Defining a Class

```
// Define a class
jshell> class Person {
   ...>     String name;
   ...>     int age;
   ...>
   ...>     void display() {
   ...>         System.out.println("Name: " + name + ", Age: " + age);
   ...>     }
   ...> }
|  created class Person

// Create an object and use it
jshell> Person person = new Person()
person ==> Person@1a2b3c4

jshell> person.name = "John"
person.name ==> "John"

jshell> person.age = 25
person.age ==> 25
```

```
jshell> person.display()
Name: John, Age: 25
```

Exiting JShell

To exit JShell, type `/exit` and press Enter.

```
jshell> /exit
|  Goodbye
```

Summary

- JShell is an interactive REPL tool for Java introduced in JDK 9.
- You can use JShell to declare variables, perform calculations, define methods, use loops and conditionals, and work with classes.
- JShell provides a convenient way to experiment with Java code and learn the language without the need for a full development environment.